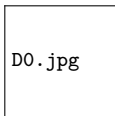


Lecture 07: Relational Model Fundamentals

Relational Model Fundamentals

Z2004: Database Management Systems



IIT Madras Zanzibar

Thursday, 12 March 2026

Today's Objectives

What You Will Learn

- ✔ **The Relational Anatomy**
Master the math vocabulary: tuples, attributes, domains, and relations.
- ✔ **The ER-to-Table Shift**
Learn how boxes and diamonds physically translate into rows and columns.
- ✔ **Relational Keys**
Differentiate between Superkeys, Candidate Keys, and Primary Keys.
- ✔ **Integrity Constraints**
Define the rules that keep database data accurate, safe, and reliable.

The Big Picture

Phase 1 is over.

We are done drawing conceptual shapes on a whiteboard.

Phase 2 begins.

Today, we translate those designs into the actual mathematical structures that a DBMS uses to store data.

"From Concepts to Implementation."

From ER Diagrams to Tables

The Paradigm Shift

ER Model: Conceptual Design
(Boxes, Diamonds, Ellipses)

Relational Model: Implementation
(Tables, Rows, Columns)

E.F. Codd (1970)

- ▶ Data is stored as **mathematical relations**.
- ▶ **Declarative:** You say *what* you want (SQL), not *how* to get it.
- ▶ Achieves **Physical Data Independence** (App doesn't care how the disk works).

The New Vocabulary

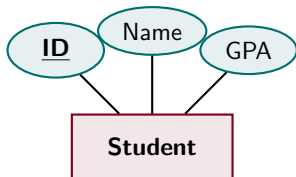
ER Entity → Relation (Table)

StudentID	Name	GPA
S001	Amina Hassan	3.8
S002	Baraka Msuya	3.2
S003	Neema John	3.9

- ▶ **Row (Tuple):** One entity instance.
- ▶ **Column (Attribute):** One property.
- ▶ **Table (Relation):** The entire entity set.
- ▶ *Note: Order of rows/cols does not matter.*

The ER-to-Table Shift: Strong Entities

1. The Conceptual Model (ER)



- ▶ **Rectangle:** The Entity Set.
- ▶ **Ovals:** The Attributes.
- ▶ **Underline:** The Unique Identifier.

2. Physical Implementation

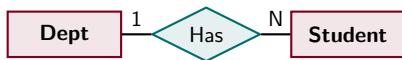
<u>ID</u>	Name	GPA
S001	Amina	3.8
S002	Baraka	3.2
S003	Neema	3.9

The Translation Rules:

- ➔ **Entity Set → Table Name**
*(We create a table called *Student*)*
- ➔ **Attributes → Columns**
(Ovals become the headers)
- ➔ **Underlined Attribute → Primary Key**
(ID becomes the PK column)

The ER-to-Table Shift: Relationships (Diamonds)

1-to-N Relationships



The Rule: **Foreign Key on the 'N' side**

- ▶ We do **not** create a new table for the diamond.
- ▶ We take the Primary Key of the '1' side (Dept) and put it inside the 'N' side (Student) as a **Foreign Key (FK)**.

Dept(DeptID, Name)
Student(StudentID, Name, DeptID_FK)

M-to-N Relationships



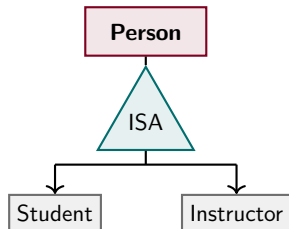
The Rule: **Create a NEW Table**

- ▶ We **must** create a new table for the diamond.
- ▶ It absorbs the Primary Keys from *both* participating entities to form a **Composite Primary Key**.

Student(StudentID, Name)
Course(CourseID, Title)
Enrolls(StudentID, CourseID)

Mapping ISA Hierarchies: Disjoint vs. Overlapping

The Inheritance Problem



Relational databases do not inherently understand "Object-Oriented Inheritance". We have to fake it using tables.

The 3 Mapping Strategies

- ▶ **1. The ER Approach (Multiple Tables)**
Create a table for the Superclass, and tables for Subclasses. Link them via FKs.
Best for: Overlapping (A Person can be in both Student and Instructor tables seamlessly).
- ▶ **2. The Object-Oriented Approach**
Only create tables for Subclasses. Inherit all superclass attributes directly into them.
Best for: Disjoint & Total. (If it's overlapping, data like 'Name' gets duplicated in both tables).
- ▶ **3. The Single Table Approach ("God Table")**
One massive Person table with a "Type" column and many NULLs for irrelevant attributes.

Advanced Mappings: Unary, Ternary, & Aggregation

Unary & Ternary Mappings

Unary (Recursive) Relationships:

- ▶ Example: Employee *manages* Employee.
- ▶ **Rule:** Add a Foreign Key that points back to its *own* Primary Key in the same table.
- ▶ Emp(EmpID, Name, **ManagerID_FK**)

Ternary Relationships (3-Way):

- ▶ Example: Doctor, Patient, Drug.
- ▶ **Rule:** Always create a **New Table**.
- ▶ The Composite PK is formed by pulling the PKs from *all three* participating entities.

Mapping Aggregation

Concept Review: We treat a Relationship as if it were an Entity, so it can participate in another relationship.

The Translation Rule:

1. Translate the "base" relationship normally (usually M:N, creating a new table with a composite PK).
2. Translate the "outer" relationship by taking the Composite PK of that base relationship and using it as a Foreign Key.

*Example: A Department **evaluates** an Instructor who teaches a Course.*

Formal Definitions: The Building Blocks

Domains & Relations

- **Domain (D):** The pool of valid, *atomic* (indivisible) values an attribute can take.
Example: D_{GPA} is all decimals from 0.0 to 5.0.
- **Tuple (t):** A single ordered row of values. Represents one real-world entity.
- **Relation (r):** A set of tuples (the table itself). Mathematically, it is a subset of the Cartesian product of its domains.

ID	Name	GPA
S001	Juma Ali	3.8
S002	Sarah Mtei	3.2
S003	Imani John	3.9

Domain (D_{GPA})

← Tuple (

Formal Definitions: Structure vs. Data

Schema vs. Instance

- **Schema (R):** The blueprint. The table name, column names, and their types.

Static: Rarely changes.

- **Instance (r):** The actual data (rows) stored at this exact second.

Dynamic: Changes with every INSERT/UPDATE.

SCHEMA		
ID	Name	GPA
S001	Juma Ali	3.8
S002	Sarah Mtei	3.2
S003	Imani John	3.9
INSTANCE		

Degree = 3 Cardinality = 3

Notation: Student(StudentID, Name, GPA)

Table Properties

- **Degree:** Number of attributes (columns).
- **Cardinality:** Number of tuples (rows).

Superkeys, Candidate Keys, and Primary Keys

The Key Hierarchy

- ▶ **Superkey (K):** A set of attributes that uniquely identifies every tuple in the relation.
It works, but may have extra attributes.
- ▶ **Candidate Key:** A *minimal* superkey.
Removing any attribute destroys uniqueness.
May have multiple.
- ▶ **Primary Key (PK):** The Candidate Key chosen by the DB Admin.
Convention: Underlined in the schema.

Keys in Action

<u>ID</u>	Name	Email	GPA
S001	Amina	amina@iitmz	3.8
S002	Baraka	baraka@iitmz	3.2
S003	Neema	neema@iitmz	3.9

- ▶ **Superkeys:** {ID}, {Email}, {ID, Name}, ...
- ▶ **Candidate Keys:** {ID}, {Email}
- ▶ **PK Chosen:** {ID}
- ▶ **Alternate Key:** {Email}
(Unique, but not picked as PK)

⚠ Entity Integrity:

Primary key attributes must **never** be NULL.

The Relational Schema Diagram

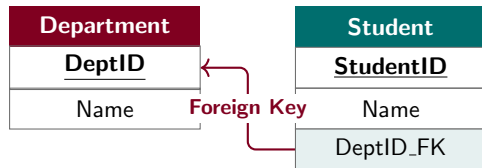
Visualizing the Database

A **Schema Diagram** is the ultimate blueprint of your implemented database. It shows how all the tables are connected.

Key Visual Elements:

- ▶ **Tables:** Represented as lists of attributes.
- ▶ **Primary Keys:** Still strictly underlined.
- ▶ **Referential Integrity (Arrows):** The most critical part. We draw an arrow from the **Foreign Key** in the referencing table pointing directly to the **Primary Key** it targets in the referenced table.

Schema in Action



Note: Unlike ER Diagrams which are for humans, Schema Diagrams represent the exact tables the DBMS will create.

University DB schema diagram

Lectures/image.png

Foreign Keys and Referential Integrity

The Foreign Key Rule

Let R_1 and R_2 be relation schemas. A set of attributes FK in R_1 is a **foreign key** referencing R_2 if:

1. The attributes in FK have the same domain (data type) as the primary key of R_2 .
2. Each value of FK in R_1 **must** either appear as a Primary Key value in R_2 , **or** be NULL.

❗ The DBMS Enforcer

Inserting an Enrolment row with $StudentID = 'S999'$ when no such student exists in the Student table violates referential integrity.

The DBMS will strictly reject the operation.

Referential Integrity in Action

Student (Referenced)

<u>StudentID</u>	Name
S001	Amina
S002	Juma

Enrolment (Referencing)

<u>EnrolID</u>	StudentID_FK
E01	S001
E02	S002
E03	S001
E04	S999

✗ Rejected ←

Schema Notation:

Student(StudentID, Name)

Enrolment(EnrolID, StudentID_FK)

Domain and Entity Integrity Constraints

Domain Constraints

Ensures that all data inside a specific column is valid and matches the expected format or business rules.

- ▶ **Data Types:** e.g., INTEGER, DATE
- ▶ **Mandatory:** NOT NULL
- ▶ **Custom Rules:** CHECK clauses

Example: CHECK (GPA >= 0.0 AND GPA <= 5.0)

Name	GPA
Amina	3.8
Juma	-1.5

✗ Rejected

Entity Integrity

Ensures that every tuple (row) in a table represents a distinct, identifiable real-world entity.

- ▶ **Rule 1:** Primary Keys must be **UNIQUE**.
- ▶ **Rule 2:** Primary Keys cannot be **NULL**.

Example: Student (ID, Name)

<u>ID</u>	Name
S001	Amina
NULL	Baraka
S001	Neema

✗ Null PK

✗ Duplicate

Properties of a Relation (Table)

The Core Rules

Mathematically, a relation is a **set**. It must follow strict rules:

1. **No duplicates:** Every row must be unique.
2. **No row ordering:** Rows have no inherent top-to-bottom order.
3. **No column ordering:** Columns have no inherent left-to-right order.
4. **Unique Attributes:** Column names must be unique within the table.
5. **Atomic values (1NF):** Each cell can hold exactly **one** indivisible value.

Atomicity (1NF) Violation

StudentID	Phone
S001	0712345, 0723456

← - ✗ Not Atomic

Fix: Move multi-valued attributes to a separate table.

Theory vs. SQL Reality

While formal algebra enforces strict sets, **SQL uses multisets**:

- ▶ SQL *allows* duplicate rows (unless using `DISTINCT` or `PK`).
- ▶ SQL allows forcing a view order using `ORDER BY`.

The Concept of NULL

The Missing Data Problem

In the real world, we don't always have all the data. The Relational Model uses **NULL** to represent this.

NULL means one of two things:

- ▶ **Unknown:** The value exists, but we don't know it yet.
e.g., A student's exam score before it is graded.
- ▶ **Inapplicable:** The attribute does not make sense for this specific tuple.
e.g., "Apartment Number" for someone who lives in a standalone house.

⊘ What NULL is NOT

NULL is NOT Zero (0).

A bank balance of 0 means you are broke. NULL means the bank doesn't know your balance.

NULL is NOT an Empty String ("").

An empty string means "I definitively have no middle name." NULL means "I didn't tell you."

NULL in Operations

Because NULL is unknown, operations evaluate to UNKNOWN:

- ▶ $5 + \text{NULL} = \text{NULL}$
- ▶ $\text{NULL} > 10 = \text{UNKNOWN}$

Key Takeaways

- ✓ **The Core Math:** A relation is a subset of a Cartesian product; rows are tuples.
- ✓ **Architecture:** Schema = Structure (Static); Instance = Current Data (Dynamic).
- ✓ **Keys:** Superkey $\supset$ Candidate Key $\supset$ Primary Key.
- ✓ **Connections:** Foreign keys link tables and enforce referential integrity.
- ✓ **Strict Rules:** Relations have no row/column order, no duplicates, and require atomic values (1NF).

Next Lecture (L08)

Deep Dive: Integrity Constraints

- ➔ Domain constraints & Data Types
- ➔ Entity integrity (Handling NULLs)
- ➔ Referential integrity complexities
- ➔ Constraint violations and ON DELETE / UPDATE actions.

Required Reading

 **Silberschatz, Chapter 2**
Introduction to the Relational Model